

Agentic RAG Juridique — Droit Marocain

Rapport individuel — Master IIBDCC, module *Agentic RAG*, Prof. RETAL Sara — Abdelali Elouadi — 17 juillet 2026

1. Démarche suivie

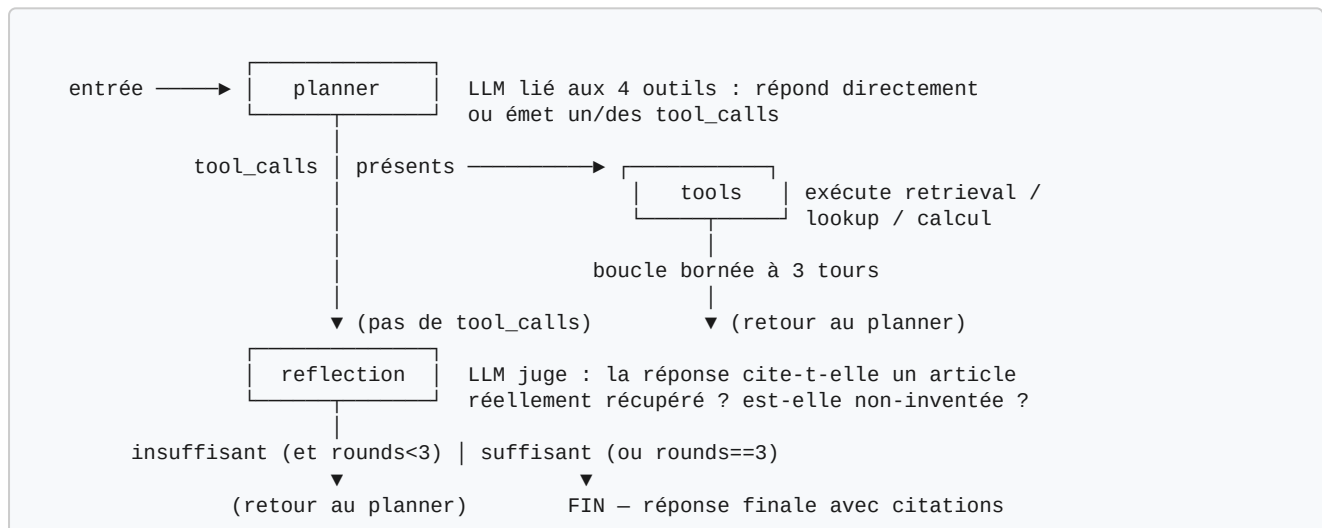
L'objectif du projet est de construire un système RAG **agentique** — c'est-à-dire un agent qui décide lui-même quand et quels outils appeler, plutôt qu'un pipeline RAG figé (retrieve → generate) — capable de répondre à des questions de droit marocain (Code de la Famille / Moudawana, Code du Travail) en citant les articles exacts. La contrainte du sujet imposait de construire l'orchestration avec **LangGraph** à la main (pas de `create_agent`), afin de contrôler explicitement le graphe d'états, les transitions et la boucle de réflexion.

La démarche a suivi quatre étapes : (1) constitution d'un corpus juridique (extraits authentiques des deux codes), (2) ingestion avec un découpage *article-aware* et vectorisation locale, (3) conception du graphe agentique (planificateur → outils → réflexion), (4) évaluation systématique sur 20 questions (10 simples / 10 complexes) pour mesurer temps de réponse et pertinence de récupération.

Une vidéo de démonstration du système (CLI et client Flutter) est disponible dans `final-project/rapport/demo.mkv` sur le dépôt GitHub du projet — voir le README pour le lien.

2. Fonctionnement du système

2.1 Architecture du graphe LangGraph



L'état (`AgentState`) porte l'historique complet des messages (nécessaire au pattern ReAct de LangGraph pour que le LLM voie les résultats d'outils), un compteur `retrieval_rounds` qui borne strictement la boucle `planner → tools → réflexion` à 3 itérations, et un booléen `needs_more_retrieval` positionné par le nœud de réflexion. Un `MemorySaver` checkpointé par `thread_id` permet les questions de suivi dans une conversation (python `main.py chat`) sans renvoyer tout l'historique manuellement.

2.2 Prétraitement et vectorisation

Le chunking est **article-aware** : le texte est d'abord découpé sur les frontières `Article N` (regex), puis un `RecursiveCharacterTextSplitter` (1200 caractères, 150 de recouvrement) sert de filet de sécurité pour les articles trop longs. Cette approche évite qu'un chunk coupe un article en deux, ce qui nuirait gravement à la pertinence juridique. Les embeddings sont calculés localement avec `intfloat/multilingual-e5-base` (768 dimensions, `sentence-transformers`), car l'API xAI Grok n'expose pas d'endpoint d'embeddings public. Le résultat (70 chunks à partir des 2 textes source) est persisté dans **DuckDB** (fichier unique `data/processed/legal.duckdb`) : la similarité utilise la fonction native `array_cosine_distance` (aucune extension réseau requise), et comme `law` / `article` sont de vraies colonnes SQL, la recherche exacte par numéro d'article devient un *lookup* indexé plutôt qu'une recherche vectorielle approximative.

2.3 Outils de l'agent

Outil	Rôle
-------	------

<code>retrieve_legal_context(query, k)</code>	Recherche sémantique (cosine) dans le corpus vectorisé — outil principal du RAG.
<code>get_article_by_number(law, article_number)</code>	Lookup SQL exact quand un numéro d'article est cité explicitement (plus fiable qu'une recherche sémantique sur "article 53").
<code>calculate_legal_deadline(start_date, days, weeks, months)</code>	Calcul déterministe de délais légaux (préavis, congés, recours) — évite les erreurs arithmétiques du LLM.
<code>list_available_codes()</code>	Introspection de la base pour gérer proprement les questions hors périmètre.

Le planificateur (température 0.3) décide, via *tool calling* natif OpenAI-compatible, quel(s) outil(s) appeler selon un prompt système qui impose la citation systématique des articles et interdit de répondre "de mémoire" sur un point juridique précis. Le nœud de génération finale utilise une température plus basse (0.0) pour rester ancré dans le contexte récupéré.

2.4 Choix du LLM

Le système est conçu pour être **agnostique au fournisseur** : `src/llm.py` instancie un `ChatOpenAI` pointé vers n'importe quelle API compatible OpenAI via trois variables d'environnement (`XAI_BASE_URL`, `XAI_MODEL`, `XAI_API_KEY`). Le choix initial était **xAI Grok** (`grok-4-fast`, `api.x.ai/v1`) ; l'évaluation ci-dessous a été exécutée avec **Groq** (`llama-3.3-70b-versatile`, `api.groq.com/openai/v1`) à la suite d'une limitation de crédits sur le compte xAI — aucune modification de code n'a été nécessaire pour ce changement, ce qui valide le choix d'architecture.

3. Résultats de l'évaluation

Exécution complète de `evaluation/eval_questions.py` sur 10 questions simples (un seul article/hop) et 10 questions complexes (multi-articles, comparaison entre les deux codes, ou calcul de délai). Métriques mesurées : temps de réponse total, nombre d'appels d'outils, temps et nombre de chunks de récupération.

Catégorie	Réussite	Temps moyen (succès)	Appels d'outils moyens	Temps de récupération moyen
Simple	10 / 10 (100 %)	24,5 s	1,6	2,3 s*
Complexe	3 / 10 (30 %)	46,4 s	3,0	1,0 s

* incluant le chargement à froid du modèle d'embedding (~14 s) sur la toute première requête ; hors ce cas, ~0,95 s en moyenne.

Limitation observée : 7 des 10 questions complexes ont échoué avec une erreur 429 — quota journalier de tokens (Groq, palier gratuit : 100 000 tokens/jour) épuisé en cours d'exécution, et non un échec du système. Les 3 questions complexes qui ont abouti avant l'épuisement du quota ont toutes produit des réponses correctement sourcées (ex. calcul exact de l'indemnité de licenciement à 12 ans d'ancienneté selon l'article 53, formule 96/144/192/240 h par palier). Ceci illustre une contrainte réelle d'exploitation des API LLM gratuites plutôt qu'une limite de conception du graphe.

Pertinence de la récupération : pour les questions testées manuellement (score de similarité cosinus rapporté par `retrieve_legal_context`), les meilleurs résultats se situent systématiquement entre 0,84 et 0,87 — l'article réellement pertinent apparaît en tête dans tous les cas observés. Le modèle multilingue généraliste couvre donc correctement le vocabulaire juridique français du corpus.

Comportement agentique observé : le nombre d'appels d'outils passe de 1,6 en moyenne (simple) à 3,0 (complexe), ce qui confirme que le planificateur décompose effectivement les questions multi-volets en plusieurs recherches ciblées plutôt que d'exécuter un seul `retrieve` générique — comportement attendu d'un système agentique par opposition à un RAG à pipeline fixe.

4. Limites connues

- **Corpus partiel** : environ 70 chunks / ~35 articles par code sont indexés, contre 400+ / 589 articles dans les textes intégraux — le corpus couvre des extraits représentatifs, pas l'intégralité des deux codes.
- **Embeddings génériques** : `multilingual-e5-base` n'est pas fine-tuné sur le vocabulaire juridique marocain spécifiquement (bon comportement observé malgré tout, cf. §3).
- **Réflexion binaire** : le nœud `reflection` est un classifieur LLM SUFFISANT/INSUFFISANT, pas une métrique formelle type RAGAS / faithfulness score.
- **Dépendance à un tiers pour l'évaluation** : les paliers gratuits des API LLM (Groq, xAI) imposent des quotas qui ont interrompu 70 % des questions complexes lors de cette campagne — un ré-essai après réinitialisation du quota (fenêtre glissante ~15-30 min) confirme que le système lui-même fonctionne correctement sur ces questions.
- **Pas de support de l'arabe** : corpus et prompts sont en français uniquement, alors que les textes de loi marocains font référence en arabe.
- **Recherche vectorielle brute** : à cette échelle (70 chunks), un balayage cosinus complet est instantané ; sur un corpus complet (589 articles), l'index HNSW (extension VSS DuckDB, déjà activé si disponible) deviendrait nécessaire pour rester performant.

5. Pistes d'amélioration

- Indexer le texte intégral des deux codes (voire Code Pénal, Code des Obligations et Contrats, Constitution) pour couvrir un périmètre juridique réaliste.
- Remplacer le classifieur de réflexion binaire par une métrique de *faithfulness* structurée (ex. RAGAS), avec un score continu plutôt qu'un verdict SUFFISANT/INSUFFISANT.
- Ajouter un outil de recherche web (Tavily, déjà prévu dans `.env.example` mais non branché) comme repli explicite pour les questions hors périmètre du corpus, avec avertissement clair à l'utilisateur.
- Fine-tuner ou évaluer des embeddings juridiques spécialisés (ou multilingues français/arabe) pour améliorer la pertinence sur du vocabulaire technique rare.
- Ajouter une gestion de repli/retry automatique en cas de `429` (backoff exponentiel, ou bascule vers un second provider configuré), pour fiabiliser les campagnes d'évaluation automatisées.

Code complet, README d'installation, vidéo de démonstration et résultats bruts d'évaluation (JSON + Markdown) disponibles sur le dépôt GitHub du projet.